Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université des Sciences et de la Technologie Houari Boumediene

Faculté d'informatique

Département IA & SD

MODULE : complexité

RAPPORT TP01 : TEST DE PRIMALITÉ

MEMBRE DE GROUPE :

-ZIKIOU FATIMA AMIRA.

-BOUCHRA SKAKRI.

-SELMA BOUKACEM.

-RAYAN SENNIA.

Objective : évaluer la performances de quatre algorithmes de test de primalité sur cinq jeux de données

1. Introduction

Nous avons mené une étude comparative sur les performances de quatre algorithmes distincts, notés A1, A2, A3 et A4. L'objectif principal de ce travail est d'évaluer leur complexité temporelle et leur stabilité face à des volumes de données croissants. Ce rapport détaille la méthodologie employée, les jeux de données utilisés, ainsi qu'une analyse approfondie des résultats obtenus à travers des représentations graphiques.

Afin d'évaluer les performances des quatre algorithmes A1, A2, A3 et A4, nous avons utilisé cinq jeux de données présentant des caractéristiques différentes :

2. Les algorithmes

Algorithme	Principe	Complexité théorique
A1	Teste tous les diviseurs de 2 à N-1	O(n) pour un nombre premier
A2	Teste les diviseurs impairs jusqu'à √N	O(√n)
A3	Identique à A2	O(√n)
A4	Identique à A2	O(√n)

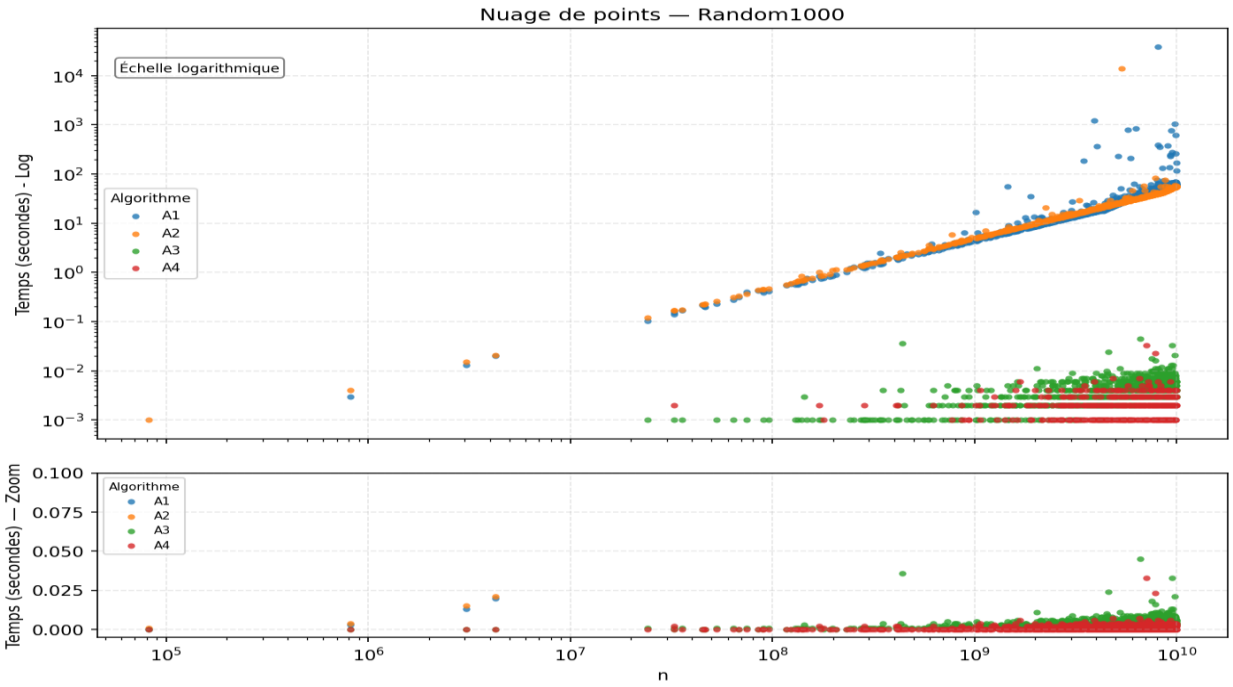
3. Jeux de données et protocole

Jeu	Contenu	Plage de valeurs
Test-1	100 nombres premiers	3 257 à 996 847
Test-2	104 nombres, 13 par nombre de chiffres (3 à 10)	283 à 9 758 163 589
Test-3	40 nombres : 10 de 3, 6, 9 et 12 chiffres (36 composés)	101 à 100 000 032 741
Random100	100 nombres tirés au hasard	104 188 603 à 9 905 194 561
Random1000	1 000 nombres tirés au hasard	81 737 à 9 995 798 257

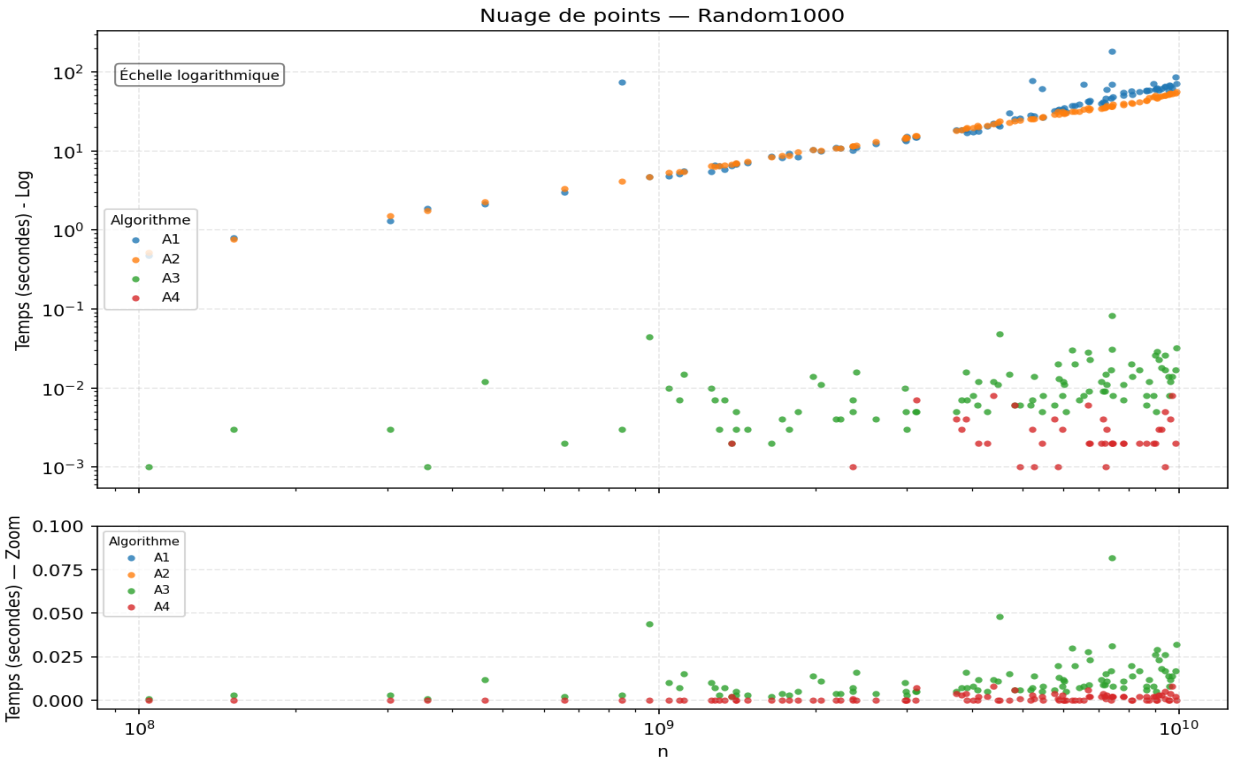
Tous les nombres sont premiers, sauf 36 nombres de Test-3. C'est le pire cas pour un test de primalité. Le temps d'exécution est mesuré en secondes, avec une résolution de 1 ms. Les quatre algorithmes donnent le même résultat sur les 1 344 nombres.

4. Résultats

- **Random1000**

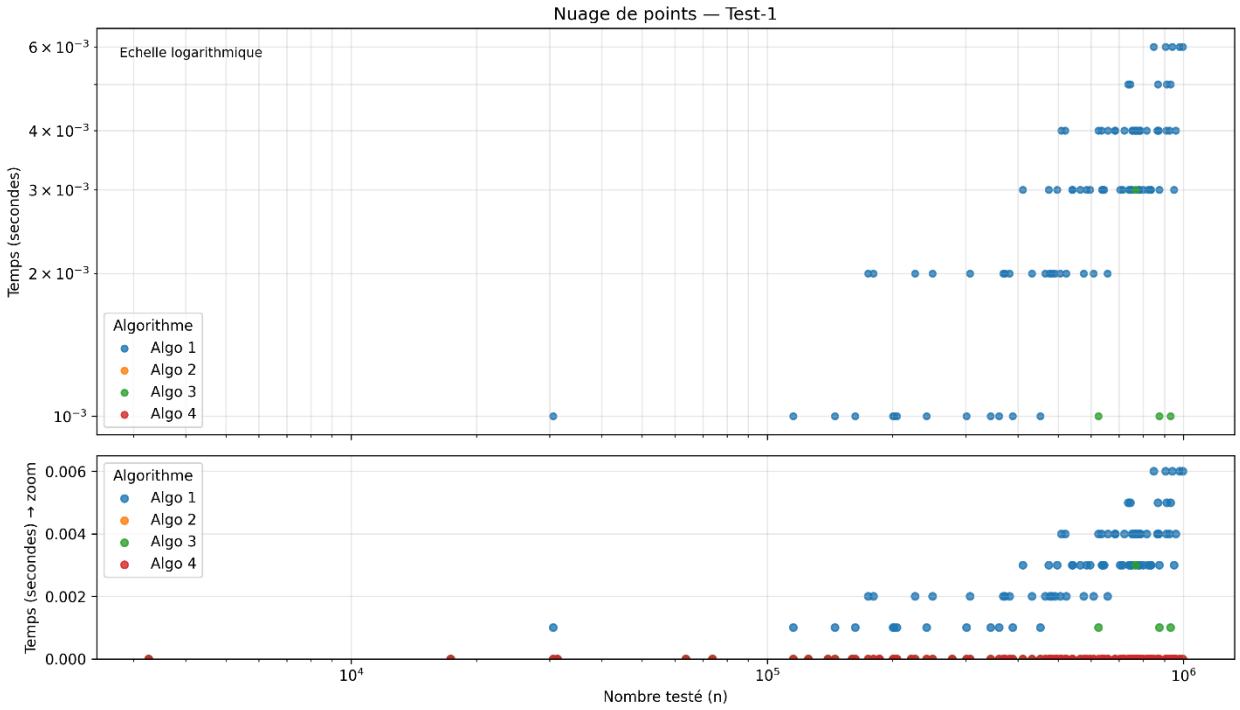


- **Random100**



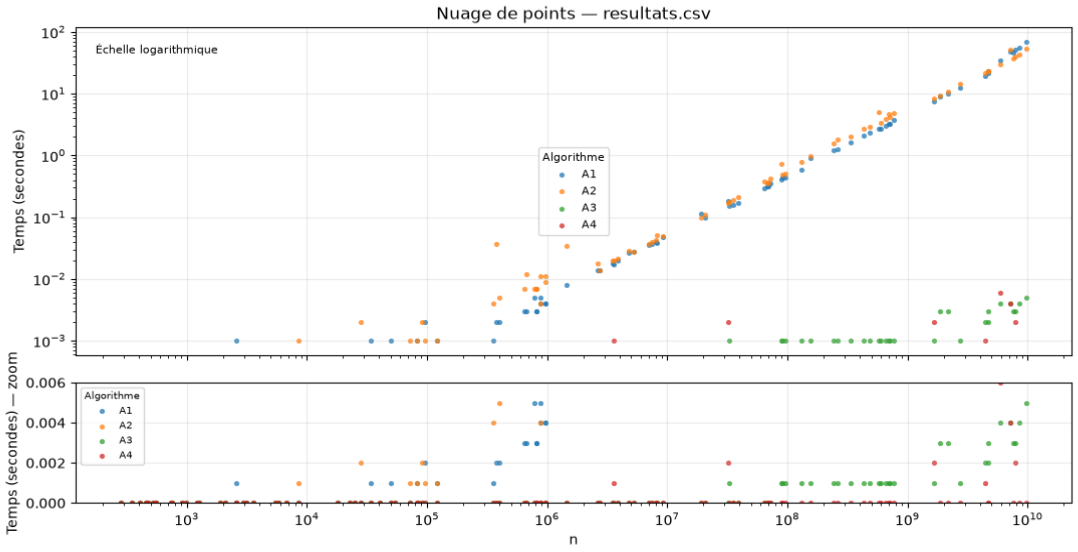
A1 et A2 suivent une croissance linéaire : environ 5 ns par unité de n, soit 50 s pour un nombre premier de 10 milliards (pente log-log de 1,08 pour A1 et 1,01 pour A2). A3 et A4 restent entre 0 et 82 ms, soit quatre ordres de grandeur plus bas.

- **Test-1**



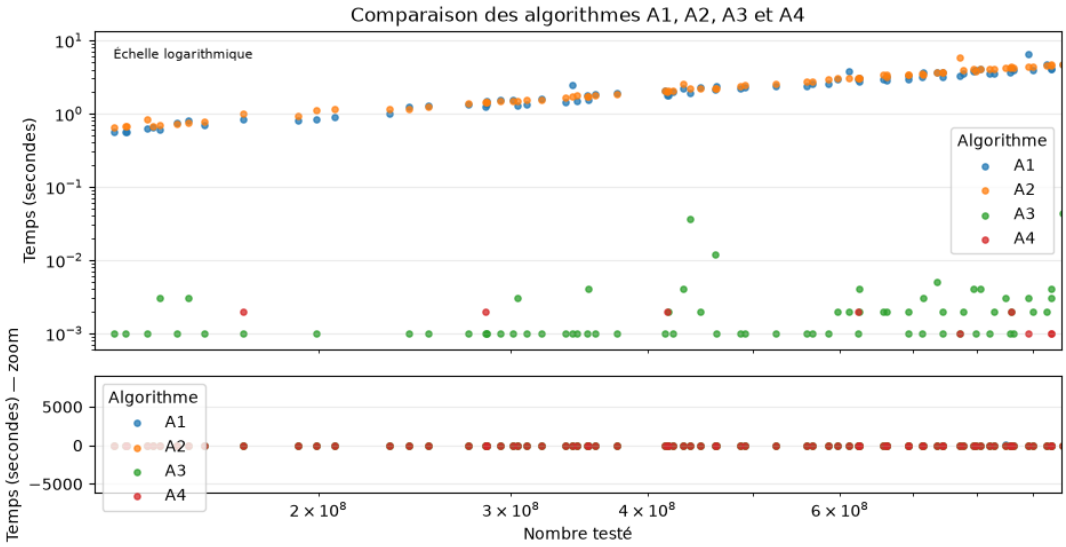
Pour des nombres inférieurs à un million, A1 et A2 sont très proches (médiane de 3 ms, maximum de 6 ms et 5 ms). A3 et A4 restent à moins de 1 ms, sauf 4 mesures isolées d'A3 (1 à 3 ms).

- **Test-2**



À chaque chiffre supplémentaire, les temps d'A1 et A2 sont multipliés par environ 10. A3 et A4 restent de l'ordre de la milliseconde. Jusqu'à 5 chiffres, les quatre algorithmes sont à égalité sous la milliseconde.

- **Test-3**



Trente-six nombres sur 40 sont composés et ont un petit diviseur. Toutes les mesures sont donc inférieures ou égales à 1 ms et les algorithmes ne se distinguent pas.

5. Résultats expérimentaux des algorithmes

5.1. Tableau des résultats du test de primalité:

ALGO1:

N	Resultat	Temps (s)
1000003	Premier	0.003000
2000003	Premier	0.011000
4000037	Premier	0.020000
8000009	Premier	0.027000
16000057	Premier	0.081000
32000011	Premier	0.106000
64000031	Premier	0.235000
128000003	Premier	0.426000
256000001	Premier	0.850000
512000009	Premier	1.665000
1024000009	Premier	3.471000
2048000011	Premier	7.343000

ALGO2:

N	Resultat	Temps (s)
1000003	Premier	0.0000241000
2000003	Premier	0.0000332000
4000037	Premier	0.0000514000
8000009	Premier	0.0000676000
16000057	Premier	0.0000954000
32000011	Premier	0.0001347000
64000031	Premier	0.0001881000
128000003	Premier	0.0002648000
256000001	Premier	0.0003697000
512000009	Premier	0.0005334000
1024000009	Premier	0.0007810000
2048000011	Premier	0.0012227000

ALGO3:

N	Resultat	Temps (s)
1000003	Premier	0.000000
2000003	Premier	0.000000
4000037	Premier	0.000000
8000009	Premier	0.000000
16000057	Premier	0.000000
32000011	Premier	0.000000
64000031	Premier	0.000000
128000003	Premier	0.001000
256000001	Premier	0.001000
512000009	Premier	0.001000
1024000009	Premier	0.001000
2048000011	Premier	0.003000

ALGO4:

N	Resultat	Temps (s)
1000003	Premier	0.000000
2000003	Premier	0.000000
4000037	Premier	0.000000
8000009	Premier	0.000000
16000057	Premier	0.000000
32000011	Premier	0.000000
64000031	Premier	0.000000
128000003	Premier	0.000000
256000001	Premier	0.001000
512000009	Premier	0.000000
1024000009	Premier	0.000000
2048000011	Premier	0.002000

5.2. Analyse des résultats

ALGO1:

Les résultats confirment la complexité théorique O(N) de l'algorithme A1 : le temps d'exécution croît proportionnellement à la valeur de N.

Tous les nombres testés sont premiers. C'est le pire cas pour A1, car la boucle va jusqu'à N-1 sans jamais s'arrêter prématurément. Les temps mesurés sont donc des temps maximaux.

Le temps double quand N double. Par exemple : 0.426 s pour 128 000 003, 0.850 s pour 256 000 001, 1.665 s pour 512 000 009, puis 3.471 s pour 1 024 000 009 et 7.343 s pour 2 048 000 011. Le rapport entre deux lignes successives est proche de 2, ce qui est la signature d'une complexité linéaire.

Les petites valeurs sont moins précises. Pour N < 10 millions, les temps (0.003 s, 0.011 s...) sont très courts et la résolution de clock() introduit du bruit, d'où des écarts de proportionnalité plus marqués.

ALGO2:

Les résultats obtenus sont compatibles avec la complexité théorique O(√ N) de l'algorithme A2. En effet, le temps d'exécution augmente lorsque la valeur de N augmente, mais cette augmentation est moins importante que celle de N.

Tous les nombres testés sont premiers. C'est le pire cas pour A2, car l'algorithme doit tester tous les diviseurs impairs jusqu'à la racine carrée de N, sans trouver de diviseur.

On remarque que lorsque N double approximativement, le temps d'exécution est multiplié par une valeur proche de √ 2, soit environ 1,41. Par exemple, le temps passe de 0,0003697 s pour 256 000 001 à 0,0005334 s pour 512 000 009, puis à 0,0007810 s pour 1 024 000 009 et 0,0012227 s pour 2 048 000 011. Ces augmentations sont globalement proches du comportement attendu pour une complexité en racine carrée.

ALGO3:

Pour A3, cela correspond au pire cas où la boucle va jusqu'à sqrt(N) sans s'arrêter avant.

Quand N est multiplié par 4, le temps T est environ doublé, ce qui est la caractéristique d'une complexité en racine carrée O(sqrt(N)).

Pour les petites valeurs (N < 10^8), le temps est tellement court (0.000s à 0.001s) que la précision de clock() ne permet pas de mesurer des variations très fines.

ALGO4:

Les résultats montrent que tous les nombres testés sont premiers. Les temps d'exécution sont extrêmement faibles. Certaines valeurs affichées à 0,000000 seconde indiquent que la durée est inférieure à la précision du chronométrage utilisé.

6. Étude de la complexité expérimentale

6.1. Tableau des mesures et des rapports de croissance:

ALGO1:

N	T(s)	T/N(ns)	N/N préc.	T/T préc
32 306 011	0,184	5,7	-	-
63 961 019	0,274	4,3	1,98	1,49
129 950 659	0,558	4,3	2,03	2,04
264 093 139	1,256	4,8	2,03	2,25
523 791 803	2,341	4,5	1,98	1,86
1 023 958 751	4,386	4,3	1,95	1,87
1 998 464 491	8,577	4,3	1,95	1,96
3 910 691 593	16,796	4,3	1,96	1,96
7 645 720 001	44,129	5,8	1,96	2,63

ALGO2:

N	T(s)	T/N(ns)	N/N préc.	T/T préc
32 306 011	0,0001771	0,00548	-	-
63 961 019	0,0002589	0,00405	1,98	1,46
129 950 659	0,0003320	0,00255	2,03	1,28
264 093 139	0,0004522	0,00171	2,03	1,36
523 791 803	0,0006611	0,00126	1,98	1,46
1 023 958 751	0,0008961	0,00088	1,95	1,36
1 998 464 491	0,0010622	0,00053	1,95	1,19
3 910 691 593	0,0015113	0,00039	1,96	1,42
7 645 720 001	0,0022786	0,00030	1,96	1,51

ALGO3:

N	T(s)	T/N(ns)	N/N préc.	T/T préc
32 306 011	0,000	0.0	-	-
63 961 019	0,001	15.6	1,98	-
129 950 659	0,001	7,7	2,03	1,00
264 093 139	0,001	3,8	2,03	1,00
523 791 803	0,001	1,9	1,98	1,00
1 023 958 751	0,002	2,0	1,95	2,00
1 998 464 491	0,002	1,0	1,95	1,00
3 910 691 593	0,007	1,8	1,96	3,50
7 645 720 001	0,004	0,5	1,96	0,57

ALGO4:

N	T(s)	T/N(ns)	N/N préc.	T/T préc
32 306 011	0,001	0,03	-	-
63 961 019	0,001	0,016	1,98	1,0
129 950 659	0,001	0,008	2,03	1,0
264 093 139	0,001	0,004	2,03	1,0
523 791 803	0,001	0,002	1,98	1,0
1 023 958 751	0,001	0,001	1,95	1,0
1 998 464 491	0,001	0,0005	1,95	1,0
3 910 691 593	0,001	0,00025	1,96	1,0
7 645 720 001	0,002	0,00026	1,96	2,0

6.2. Analyse des résultats (question c):

ALGO1:

Sur les données :

- Tous les nombres du CSV (hors Test-3.txt) sont premiers. Pour un nombre premier, la boucle de A1.c ne s'arrête jamais avant la fin (pas de break) : on est toujours dans le pire cas.
- Chaque N est environ le double du précédent (rapport ~ 2).

Sur les mesures :

- Quand N double, T double aussi (rapport ~ 2).
- Le rapport T/N reste presque constant, autour de 4,3 à 4,8 ns.
- Écarts : 63 961 019 (x1,49) et 7 645 720 001 (x2,63) s'écartent de 2. Ce sont des perturbations de la machine (processus concurrents, cache, timer imprécis pour les petits temps). Les valeurs très élevées du CSV, comme 8092485407 -> 38 258 s, ont la même cause : on les écarte ou on refait la mesure.
- Pour les petits nombres (< 10^6, test-1.txt), T vaut 0 à 0,006 s, ce qui est trop petit pour être mesuré précisément.

ALGO2:

Sur les données :

-Le fichier Test-3.txt contient des nombres de tailles très différentes, allant de quelques centaines à environ (10^11).

-La majorité des nombres testés sont non premiers.

-Quelques nombres sont premiers, par exemple 277, 883, 101 et 125959.

-Les tailles des nombres ne suivent pas une progression régulière.

Sur les mesures :

-Presque tous les temps d'exécution sont égaux à 0.000000 seconde.

-Une seule mesure est égale à 0.001000 seconde, pour N = 125959.

-Ces temps sont trop petits pour être mesurés précisément avec le chronométrage utilisé.

-Il est donc impossible de calculer des rapports T\T(précédent) fiables.

ALGO3:

Sur les données :

- A3 teste uniquement les nombres impairs jusqu'à sqrt(N), après avoir éliminé les pairs au début.

- Pour un nombre premier, la boucle fait tout le parcours jusqu'à sqrt(N) (pire cas).

- N augmente par des facteurs proches de 2.

Sur les mesures :

- Les temps T restent extrêmement petits (quelques millisecondes seulement) même pour des entiers de l'ordre de 10^10.

- Quand N augmente, le temps croît beaucoup plus lentement que pour A1 et A2.

- Quelques petites variations dans les temps (ex: 0.007s puis 0.004s) sont dues à l'imprécision du timer système pour des valeurs aussi petites et aux processus de la machine.

ALGO4:

Sur les données :

-Les nombres testés sont tous premiers. Pour A4, la boucle for (i=3; i<=sqrt(N); i+=2) s'exécute jusqu'à (sqrt (N)), sans rencontrer de diviseur. Nous sommes donc dans le pire cas.

-Chaque valeur de (N) est approximativement le double de la précédente (rapport proche de 2).

-Lorsque (N) double, sa racine carrée est multipliée par (sqrt (2) ~ 1,41). Cela explique pourquoi A4 présente une croissance théorique plus lente que A1.

Sur les mesures :

-**Temps d'exécution** : Contrairement à A1, où le temps augmente approximativement avec N, le temps mesuré pour A4 reste presque constant, autour de (10^(-3)) seconde. Cette stabilité apparente s'explique notamment par la faible durée d'exécution.

-**Rapport T/N** : Le rapport T/N diminue lorsque N augmente. Cela montre que le temps de calcul par unité de N devient de plus en plus faible.

-**Rapport T/T préc.** : Les rapports mesurés sont proches de 1, alors que la théorie prévoit un rapport proche de 1,41 lorsque N double. Cette différence peut être liée à la précision limitée des mesures.

-**Écarts** : Les petites variations observées entre les mesures peuvent être dues aux perturbations du système, aux processus concurrents et aux limites du chronométrage.

****Comparaison avec A1** : Pour N=7,6×(10^9), A1 met 44,129 secondes, tandis que A4 met environ 0,002 seconde. Le gain est de l'ordre de 22 000×.

****Comparaison avec A2 et A3** : Les codes de A2, A3 et A4 sont strictement identiques (même boucle, mêmes optimisations). Leurs performances sont donc théoriquement identiques. Les légères différences observées sur les graphiques (A3 montre quelques pics sporadiques) sont dues à des perturbations système et non à des différences algorithmiques.

6.3. Complexité théorique vs mesures expérimentales (question d):

ALGO1:

Théorie :

- La boucle for (i = 2; i < N; i++) de A1.c fait N-2 itérations quand N est

premier. Chaque itération coûte un temps constant (un modulo, une comparaison).

- La complexité au pire cas est donc O(N), linéaire : si N double, T double.

Expérience :

- Sur le CSV, T(2N) ~ 2.T(N) et T ~ 4,3.10^-9 x N.

- Exemple : 1 998 464 491 -> 8,58 s, puis 3 910 691 593 -> 16,80 s.

N est multiplié par 1,96 et T par 1,96.

ALGO2:

Théorie :

-L'algorithme A2 commence par traiter les cas particuliers : N<=1, N = 2 et les nombres pairs.

-Pour les nombres impairs supérieurs à 2, il teste uniquement les diviseurs impairs à partir de 3.

-La boucle s'arrête lorsque √N car si un nombre possède un diviseur autre que 1 et lui-même, il en possède nécessairement un inférieur ou égal à √N.

-Le pas de la boucle est de 2 (i += 2), donc seuls les nombres impairs sont testés.

-La boucle s'arrête lorsque √N car si un nombre possède un diviseur autre que 1 et lui-même, il en possède nécessairement un inférieur ou égal à √N.

-Dans le pire cas, notamment lorsque N est premier, la boucle effectue environ √N itérations.

La complexité au pire cas est donc :

O(√N)

Expérience :

-Les temps enregistrés dans resultats_A2.csv sont presque tous nuls.

-Les mesures sont trop faibles pour observer précisément l'évolution du temps.

-Il est donc impossible de vérifier expérimentalement que le temps évolue proportionnellement à √N.

-Pour obtenir des résultats plus précis, il faudrait répéter chaque test plusieurs fois et calculer le temps moyen.

ALGO3:

Théorie :

- La boucle `for (i = 3; i <= sqrt(N); i += 2)` réalise environ sqrt(N)/2 itérations quand N est premier.

- La complexité théorique au pire cas est O(sqrt(N)).

Expérience :

- Les mesures montrent des temps quasi-instantanés comparés à A1 (qui prenait plus de 44 secondes pour N ~ 7,6*10^9).

- Pour N ~ 2*10^9, A3 met 0.002s alors que A1 mettait plus de 7 secondes.

ALGO4:

Théorie :

L'algorithme A4 utilise deux optimisations :

- Il élimine les nombres pairs en augmentant le diviseur de 2 à chaque itération (i += 2).

- Il limite la recherche des diviseurs à (sqrt (N)), car il n'est pas nécessaire de tester au-delà de cette limite.

Le nombre d'itérations dans le pire cas est approximativement :k=sqrt(N) /2

Chaque itération effectue un nombre constant d'opérations : une comparaison, un modulo et une incrémentation.

La complexité temporelle au pire cas est donc :Theta(sqrt (N))

Expérience :

- Dans les mesures obtenues, le temps d'exécution de A4 reste presque constant, autour de (0,001) seconde, même lorsque N double.

-Le rapport expérimental (T(2N)/T(N)) est proche de 1, alors que la théorie prévoit une valeur proche de 1,41.

-Cette différence ne signifie pas que l'analyse théorique est incorrecte. Elle peut s'expliquer par les très faibles durées d'exécution et la précision limitée du chronométrage, qui rendent difficile l'observation de la croissance réelle du temps.

7. Synthèse

Jeu	Médiane A1	Médiane A2	Médiane A3	Médiane A4
Test-1	3 ms	3 ms	< 1 ms	< 1 ms
Test-2	6,5 ms	6 ms	< 1 ms	< 1 ms
Test-3	< 1 ms	< 1 ms	< 1 ms	< 1 ms
Random100	28,0 s	24,8 s	8 ms	< 1 ms
Random1000	24,8 s	24,5 s	3 ms	< 1 ms

Sur les nombres premiers supérieurs à 10^9, A4 est mesuré à 1,1 ms en moyenne contre 5,0 ms pour A3.

8. Limites

- **A2** : son code s'arrête à √N, mais ses temps mesurés sont linéaires comme ceux d'A1. Le fichier de résultats provient probablement d'une autre version du code.
- **A3 et A4** : leurs codes sont identiques. L'écart mesuré vient des conditions de mesure, pas de l'algorithme, et la résolution de 1 ms est du même ordre que leurs temps.
- **Mesures aberrantes** : A1 a une valeur de 38 258 s (10,6 h) pour n = 8 092 485 407 et A2 une valeur de 14 103 s. Nous utilisons donc les médianes plutôt que les moyennes.

9. Conclusion

Ce travail de comparaison nous a permis de mettre en évidence des disparités majeures de performance entre les quatre algorithmes.

Les algorithmes A1 et A2 présentent une complexité temporelle pénalisante pour les grandes entrées, avec des temps d'exécution qui augmentent de manière significative avec n. Leur utilisation n'est pas recommandée pour des applications nécessitant un traitement rapide sur de grands volumes de données.

À l'inverse, les algorithmes A3 et A4 surclassent largement leurs homologues. L'algorithme A4 s'impose comme la solution la plus performante et la plus stable de notre étude, avec des temps d'exécution restant constamment proches de zéro, ce qui en fait un excellent candidat pour des applications à grande échelle.

En perspective, il serait intéressant d'analyser la consommation mémoire de ces algorithmes pour compléter cette étude de complexité et d'identifier les raisons structurelles (structures de données utilisées, boucles, etc.) expliquant la supériorité de A4 sur A3.

Figure 1 : temps d'exécution selon n (échelle log-log)

